

DBMP project on
**E-commerce Management System For Online Shopping &
Order Management**

Submitted in partial fulfilment for the degree of Bachelor of
Technology in Artificial Intelligence

Submitted by
Hetvi Dobaria (14)
Arya Gawade (17)
Prachi Gawali (18)

Under the guidance of
Prof. Kumud Wasnik



Usha Mittal Institute of Technology
Juhu Tara road, Santacruz-West
Mumbai, Maharashtra

CERTIFICATE

This is to certify that **Hetvi Dobaria, Arya Gawde, Prachi Gawali** has completed the mini project report on the topic” **E-commerce Management System For Online Shopping & Order Management**” satisfactorily in partial fulfillment for the Bachelor’s Degree in **Artificial Intelligence** under the guidance of **Prof. Kumud Wasnik** during the year 2025-2026 as prescribed by Usha Mittal Institute Of Technology SNDT Women’s University.

Guide

Prof. Kumud Wasnik

Head Of Department

Dr. Anita Morey

Principal

Dr.Yogesh Nerkar

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. we also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Abstract

The E-commerce database system is designed to efficiently manage and organize data related to online shopping activities such as users, products, orders, payments, and delivery tracking. The system uses Database Management System (DBMS) concepts to ensure structured data storage, reduce redundancy, and maintain data consistency. An Entity-Relationship (ER) model is used to represent the logical design of the database, defining entities and their relationships. The system supports key operations like user management, product handling, order processing, cart management, and payment transactions. It also includes features for tracking order status and managing delivery addresses. By applying normalization techniques and integrity constraints, the database ensures accurate and reliable data handling.

Additionally, the system is designed to handle multiple users and transactions simultaneously, ensuring smooth performance and scalability. It provides quick data retrieval and efficient query processing, which enhances overall system responsiveness. The database structure is flexible and can be extended to include additional features such as discounts, reviews, and inventory management. Overall, the system improves performance, enhances user experience, and provides a scalable solution for real-world E-commerce applications.

Keywords

E-commerce, DBMS, Database Design, ER Diagram, Normalization, Data Integrity, Relational Database, Order Management, Payment System

Sr.No.	Topic	Pg.No.	Signature
1	Introduction	6	
2	Problem Statement	7	
3	Objectives	8	
4	Relational Database Design	9	
5	Requirement Analysis	11	
6	Er Diagram	12	
7	Schema Diagram	13	
8	PostgreSQL Queries	15	
9	Conclusion	29	

1.Introduction

In the modern digital era, the growth of internet technologies has transformed the way businesses operate and customers interact with services. E-commerce systems have become an essential platform for buying and selling products online, offering convenience and accessibility. With the increasing number of online transactions, there is a need for an efficient system to manage large amounts of data, which is achieved using a Database Management System (DBMS).

A DBMS is software that allows users to create, maintain, and manage databases efficiently. It helps in ensuring data integrity, reducing redundancy, and enabling fast data retrieval. In an E-commerce system, the DBMS manages important data such as customer details, product information, orders, payments, and delivery tracking, making the system organized and reliable.

This project focuses on designing an E-commerce database using the Entity-Relationship (ER) model. The ER model helps represent entities like User, Product, Order, Cart, Payment, Address, Tracking Details, and Product Category, along with the relationships between them. For example, a user can place multiple orders, and each order can include multiple products, showing real-world relationships.

The system also manages functionalities like adding items to the cart, processing payments, storing delivery addresses, and tracking orders. Proper database design techniques such as normalization, keys, and constraints are applied to maintain consistency and avoid data duplication.

Overall, this E-commerce database system provides an efficient way to store, manage, and retrieve data. It improves system performance, ensures data accuracy, and can be used in real-world applications like online shopping platforms and retail systems.

2.Problem Statement

With the rapid growth of E-commerce platforms, managing large volumes of data related to users, products, orders, and transactions has become complex. Poorly structured systems often lead to issues like data redundancy, inconsistency, and slow data retrieval, which affect overall system performance.

The main problem addressed in this project is to design an efficient and reliable database system that can handle complex relationships between entities such as users, products, orders, and payments. The system aims to ensure data integrity, reduce duplication, and support fast and smooth operations, ultimately improving accuracy and user experience in an online shopping environment.

Key Issues

1. **Data Redundancy**
Repetition of the same data in multiple places increases storage usage and leads to inconsistency.
2. **Data Inconsistency**
When the same data exists in different locations, updates may not be reflected everywhere, causing mismatched information.
3. **Complex Relationships Management**
Managing relationships such as many-to-many (e.g., orders and products) can be difficult without proper design.
4. **Poor Data Retrieval Speed**
Inefficient database structure can slow down queries, affecting system performance.
5. **Lack of Data Integrity**
Without constraints and proper keys, incorrect or duplicate data entries may occur.

Impact on Operations

1. **Delayed Order Processing**
Inefficient data handling can slow down order placement and confirmation processes.
2. **Incorrect Information Display**
Users may see wrong product details, prices, or availability due to inconsistent data.
3. **Poor User Experience**
Slow system performance and errors reduce customer satisfaction and trust.
4. **Increased Operational Costs**
Redundant data and inefficient systems require more storage and maintenance.
5. **Difficulty in Tracking Orders**
Without proper tracking mechanisms, monitoring order status becomes challenging.

3.Objectives

The primary objective of this project is to develop an efficient and user-friendly online shopping system that simplifies the purchasing process. It aims to automate product management and inventory tracking while ensuring accurate and up-to-date data. The system provides secure user authentication and transaction handling to protect user information. It also enables real-time order tracking for a better customer experience. Overall, the project enhances efficiency, reduces manual effort, and ensures a reliable e-commerce platform.

Key Objectives

1. **Database Design**
To create a clear and organized database structure using tables, attributes, and relationships that can efficiently manage large amounts of E-commerce data such as users, products, orders, and transactions.
2. **Reduce Redundancy**
To eliminate duplicate data by applying normalization techniques, ensuring that data is stored only once and remains consistent throughout the database.
3. **Relationship Management**
To establish proper relationships like one-to-many and many-to-many between entities such as users, products, and orders for accurate data linking.
4. **Data Integrity**
To maintain accuracy and validity of data using primary keys, foreign keys, and constraints, preventing errors and duplication.
5. **Performance Improvement**
To enhance system performance by enabling fast, efficient, and accurate data retrieval for smooth operation of the system.

4. Relational database

1. Database Tables

Users Table

- **Attributes:** u_id, name, email
- **Purpose:** Stores customer details
- **Key Field:** u_id (Primary Key)

Address Table

- **Attributes:** a_id, u_id, city, state, country
- **Purpose:** Stores user address details
- **Key Field:** a_id (Primary Key), u_id (Foreign Key)

ProductCategory Table

- **Attributes:** c_id, name
- **Purpose:** Stores product categories
- **Key Field:** c_id (Primary Key)

Product Table

- **Attributes:** p_id, name, price, description, c_id
- **Purpose:** Stores product details
- **Key Field:** p_id (Primary Key), c_id (Foreign Key)

Cart Table

- **Attributes:** cart_id, u_id
- **Purpose:** Stores user shopping cart
- **Key Field:** cart_id (Primary Key), u_id (Foreign Key)

Cart_Product Table

- **Attributes:** cart_id, p_id, quantity
- **Purpose:** Represents many-to-many relationship between cart and products
- **Key Fields:** cart_id (Foreign Key), p_id (Foreign Key)

Orders Table

- **Attributes:** order_no, u_id, a_id, order_amount, order_date
- **Purpose:** Stores order details of users
- **Key Field:** order_no (Primary Key), u_id (Foreign Key), a_id (Foreign Key)

Order_Product Table

- **Attributes:** order_no, p_id, quantity
- **Purpose:** Represents many-to-many relationship between orders and products
- **Key Fields:** order_no (Foreign Key), p_id (Foreign Key)

Payment Table

- **Attributes:** payment_id, order_no, method, amount
- **Purpose:** Stores payment details for each order
- **Key Field:** payment_id (Primary Key), order_no (Foreign Key)

TrackingDetail Table

- **Attributes:** t_id, order_no, status
- **Purpose:** Tracks order delivery status
- **Key Field:** t_id (Primary Key), order_no (Foreign Key)

2. Data Integrity and Relationships

- **Primary Keys (PK):**
Each table has a unique identifier (e.g., u_id, p_id, order_no).
- **Foreign Keys (FK):**
Ensure referential integrity between tables:
 - Orders → Users
 - Orders → Address
 - Product → ProductCategory
 - Cart → Users
 - Cart_Product → Cart & Product
 - Order_Product → Orders & Product
 - Payment → Orders
 - TrackingDetail → Orders
- **Many-to-Many Relationships:**
 - Orders ↔ Products (via Order_Product)
 - Cart ↔ Products (via Cart_Product)
- **Computed Fields:**
 - Total order amount can be calculated using product price × quantity

3. Normalization

The database is normalized up to **Third Normal Form (3NF)**:

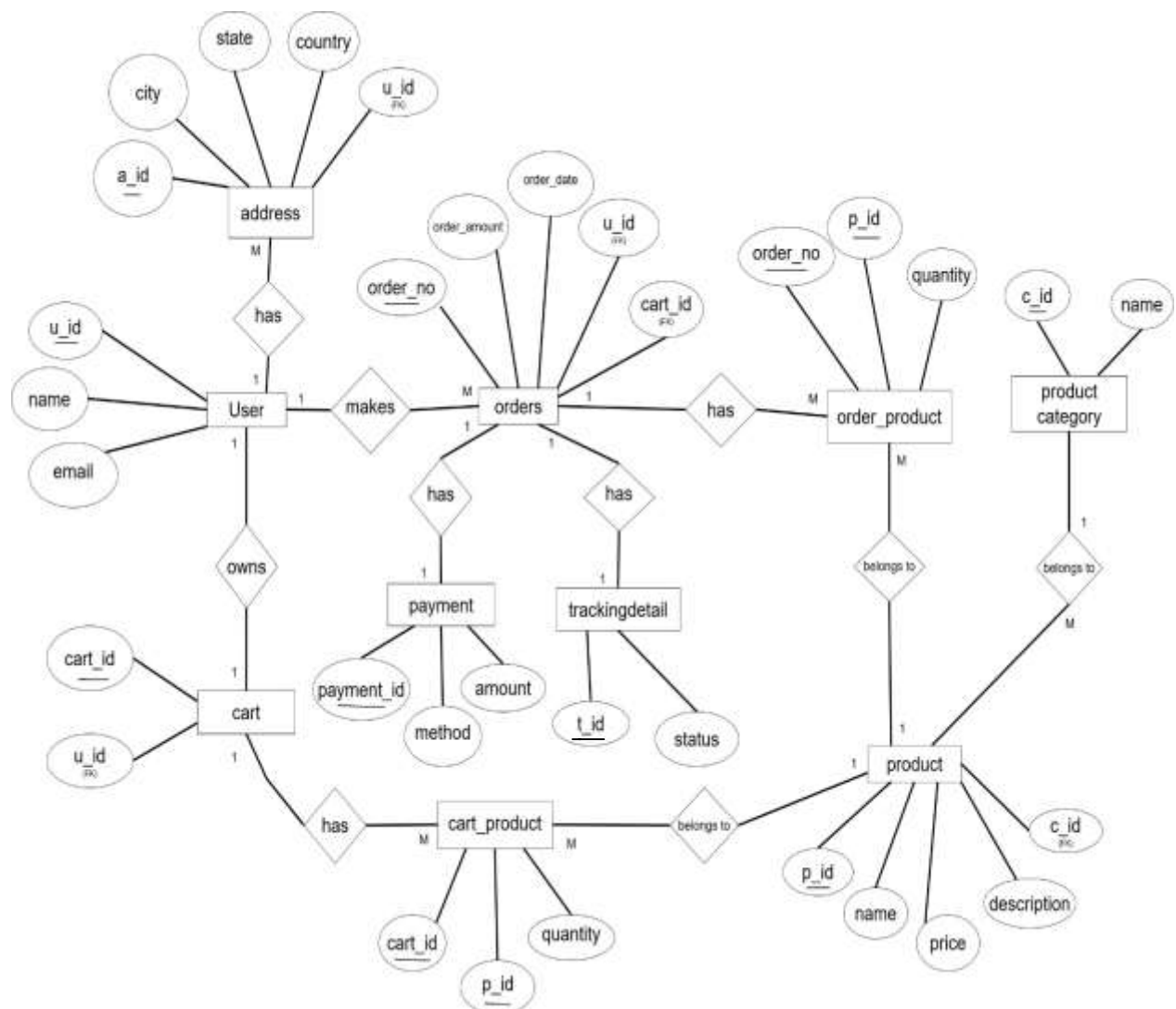
- **1NF (First Normal Form):**
All attributes are atomic (no repeating groups)
- **2NF (Second Normal Form):**
All non-key attributes depend fully on the primary key
- **3NF (Third Normal Form):**
No transitive dependency (non-key attributes depend only on primary key)

5. Requirement Analysis

The system is designed to manage E-commerce operations efficiently by handling data related to users, products, carts, orders, payments, and tracking using structured database queries.

- **User:** In the User table, we have attributes like u_id, name, and email. Out of them, u_id is the primary key because every user has a unique ID for identification.
- **Address:** In the Address table, we have attributes like a_id, u_id, city, state, and country. Out of them, a_id is the primary key. u_id is the foreign key as it links address to the User table.
- **ProductCategory:** In the ProductCategory table, we have attributes like c_id and name. Out of them, c_id is the primary key because each category has a unique ID.
- **Product:** In the Product table, we have attributes like p_id, name, price, description, and c_id. Out of them, p_id is the primary key. c_id is the foreign key as it connects product to its category.
- **Cart:** In the Cart table, we have attributes like cart_id and u_id. Out of them, cart_id is the primary key. u_id is the foreign key as each cart belongs to a specific user.
- **Cart_Product:** In this table, we have attributes like cart_id, p_id, and quantity. Here, cart_id and p_id together form a composite primary key. Both are also foreign keys linking Cart and Product tables.
- **Orders:** In the Orders table, we have attributes like order_no, u_id, cart_id, order_amount, and order_date. Out of them, order_no is the primary key. u_id and cart_id are foreign keys linking User and Cart.
- **Order_Product:** In this table, we have attributes like order_no, p_id, and quantity. Here, order_no and p_id together form a composite primary key. Both are also foreign keys linking Orders and Product.
- **Payment:** In the Payment table, we have attributes like pay_id, order_no, method, and amount. Out of them, pay_id is the primary key. order_no is the foreign key connecting to Orders.
- **TrackingDetail:** In the TrackingDetail table, we have attributes like tracking_id, order_no, and status. Out of them, tracking_id is the primary key. order_no is the foreign key linking to Orders.

6.ER diagram



7.Schema Diagram

1. User:

U_id(Pk)	name	email

2. Address:

a_id(Pk)	U_id(FK)	city	state	Country

3. Product-category:

c_id (PK)	name

4. Product:

p_id (PK)	name	price	description	c_id (FK)

5. Cart:

cart_id (PK)	u_id (FK)

6. Cart_product:

cart_id (PK, FK)	p_id (PK, FK)	quantity

7. Orders:

order_no (PK)	u_id (FK)	a_id (FK)	order_amount	Order_date

8. Order-Product:

Order_no(PF,FK)	P_id(PK,FK)	Quantity

9. Payment:

Payment_id(PK)	Order_no(FK)	a_id(FK)	amount

10.TrackingDetail:

t_id(PK)	Order_no(FK)	status

8.PostgreSQL queries

View login and password command

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:

psql (18.3)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

postgres=# \c
You are now connected to database "postgres" as user "postgres".
postgres=#
```

Create table users

```
postgres=# \c
You are now connected to database "postgres" as user "postgres".
postgres=# create table users (u_id int primary key, name varchar(50), email varchar(100));
CREATE TABLE
postgres=# insert into users values (1,'Prachi','prachi@gmail.com');
INSERT 0 1
postgres=# insert into users values (2,'Arya','arya@gmail.com');
INSERT 0 1
postgres=# insert into users values (3,'Hetvi','hetvi@gmail.com');
INSERT 0 1
postgres=# insert into users values (4,'Priya','priya@gmail.com');
INSERT 0 1
postgres=# insert into users values (5,'Shreya','Shreya@gmail.com');
INSERT 0 1
postgres=# select * from users;
 u_id | name | email
-----+-----+-----
 1 | Prachi | prachi@gmail.com
 2 | Arya | arya@gmail.com
 3 | Hetvi | hetvi@gmail.com
 4 | Priya | priya@gmail.com
 5 | Shreya | Shreya@gmail.com
(5 rows)
```

Create table address

```
postgres=# create table address (a_id int primary key, u_id int, city varchar(50), state varchar(50), country varchar(50), foreign key (u_id) references users(u_id));
CREATE TABLE
postgres=# insert into address values (1,1,'Mumbai','Maharashtra','India');
INSERT 0 1
postgres=# insert into address values (2,2,'Delhi','Delhi','India');
INSERT 0 1
postgres=# insert into address values (3,3,'Pune','Maharashtra','India');
INSERT 0 1
postgres=# insert into address values (4,4,'Lucknow','Uttar Pradesh','India');
INSERT 0 1
postgres=# insert into address values (5,5,'Bengaluru','Karnataka','India');
INSERT 0 1
postgres=# select * from address;
 a_id | u_id | city | state | country
-----+-----+-----+-----+-----
 1 | 1 | Mumbai | Maharashtra | India
 2 | 2 | Delhi | Delhi | India
 3 | 3 | Pune | Maharashtra | India
 4 | 4 | Lucknow | Uttar Pradesh | India
 5 | 5 | Bengaluru | Karnataka | India
(5 rows)
```

Create table Product-category

```
postgres=# create table productCategory (c_id int primary key, name varchar(100));
CREATE TABLE
postgres=# insert into productCategory values (1,'Electronics');
INSERT 0 1
postgres=# insert into productCategory values (2,'Clothing');
INSERT 0 1
postgres=# insert into productCategory values (3,'Footwear');
INSERT 0 1
postgres=# select * from productCategory;
 c_id |      name
-----+-----
  1   | Electronics
  2   | Clothing
  3   | Footwear
(3 rows)
```

Create table Product

```
postgres=# create table product (p_id int primary key, name varchar(50), price decimal(18,2), description varchar(100), c_id int, foreign key (c_id) references productCategory(c_id));
CREATE TABLE
postgres=# insert into product values (1,'Laptop','50000','Gaming Laptop',1);
INSERT 0 1
postgres=# insert into product values (2,'Phone','20000','Smartphone',1);
INSERT 0 1
postgres=# insert into product values (3,'T-shirt','5000','Cotton',2);
INSERT 0 1
postgres=# insert into product values (4,'Headphone','2000','Wireless',1);
INSERT 0 1
postgres=# select * from product;
 p_id | name      | price  | description | c_id
-----+-----+-----+-----+-----
  1   | Laptop   | 50000.00 | Gaming Laptop | 1
  2   | Phone    | 20000.00 | Smartphone   | 1
  3   | T-shirt  |  5000.00 | Cotton      | 2
  4   | Headphone | 2000.00 | Wireless    | 1
(4 rows)
```

Create table Cart

```
postgres=# create table cart (cart_id int primary key, u_id int, foreign key (u_id) references users(u_id));
CREATE TABLE
postgres=# insert into cart values (1,1),(2,2),(3,3);
INSERT 0 3
postgres=# select * from cart;
 cart_id | u_id
-----+-----
  1      | 1
  2      | 2
  3      | 3
(3 rows)
```

Create table Cart-Product

```
postgres=# create table cart_product (cart_id int, p_id int, quantity int, primary key (cart_id, p_id), foreign key (cart_id) references cart(cart_id), foreign key (p_id) references product(p_id));
CREATE TABLE
postgres=# insert into cart_product values (1,1,1),(1,2,2),(2,3,3),(3,4,1);
INSERT 0 4
postgres=# select * from cart_product;
 cart_id | p_id | quantity
-----+-----+-----
  1      | 1    | 1
  1      | 2    | 2
  2      | 3    | 3
  3      | 4    | 1
(4 rows)
```


Create table Orders

```
postgres=# create table orders (order_no int primary key, u_id int, a_id int, order_amount decimal(10,2), order_date DATE, foreign key (u_id) references users(u_id), foreign key (a_id) references address(a_id));
CREATE TABLE
postgres=# insert into orders values (101,1,1,90000,'2026-03-01');
INSERT 0 1
postgres=# insert into orders values (102,2,2,1500,'2026-03-02');
INSERT 0 1
postgres=# insert into orders values (103,3,3,200,'2026-03-03');
INSERT 0 1
postgres=# select * from orders;
 order_no | u_id | a_id | order_amount | order_date
-----+-----+-----+-----+-----
      101 |    1 |    1 |    90000.00 | 2026-03-01
      102 |    2 |    2 |     1500.00 | 2026-03-02
      103 |    3 |    3 |       200.00 | 2026-03-03
(3 rows)
```

Create table Order-Product

```
postgres=# create table order_product (order_no int, p_id int, quantity int, primary key (order_no, p_id), foreign key (order_no) references orders(order_no), foreign key (p_id) references product(p_id));
CREATE TABLE
postgres=# insert into order_product values (101,1,1),(101,2,2),(102,3,3),(103,4,1);
ERROR: syntax error at or near "inu"
LINE 1: insert into order_product values (101,1,1),(101,2,2),(102,3,...
              ^
postgres=# insert into order_product values (101,1,1),(101,2,2),(102,3,3),(103,4,1);
INSERT 0 4
postgres=# select * from order_product;
 order_no | p_id | quantity
-----+-----+-----
      101 |    1 |         1
      101 |    2 |         2
      102 |    3 |         3
      103 |    4 |         1
(4 rows)
```

Create table Payment

```
postgres=# create table payment (payment_id int primary key, order_no int, method varchar(50), amount decimal(10,2), foreign key (order_no) references orders(order_no));
CREATE TABLE
postgres=# insert into payment values (1,101,'UPI',90000);
INSERT 0 1
postgres=# insert into payment values (2,102,'cart',1500);
INSERT 0 1
postgres=# insert into payment values (3,10,'cash',2000);
ERROR: insert or update on table "payment" violates foreign key constraint "payment_order_no_fkey"
DETAIL: Key (order_no)=(10) is not present in table "orders".
postgres=# insert into payment values (3,103,'cash',2000);
INSERT 0 1
postgres=# select * from payment;
 payment_id | order_no | method | amount
-----+-----+-----+-----
          1 |      101 | UPI    | 90000.00
          2 |      102 | cart   |  1500.00
          3 |      103 | cash   |   2000.00
(3 rows)
```

Create table Tracking Detail

```
postgres=# create table TrackingDetail (t_id int primary key, order_no int, status varchar(20), foreign key (order_no) references orders(order_no));
CREATE TABLE
postgres=# insert into TrackingDetail values (1,101,'shipped');
INSERT 0 1
postgres=# insert into TrackingDetail values (2,102,'Delivered');
INSERT 0 1
postgres=# insert into TrackingDetail values (3,103,'Processing');
INSERT 0 1
postgres=# select * from TrackingDetail;
 t_id | order_no | status
-----+-----+-----
     1 |      101 | shipped
     2 |      102 | Delivered
     3 |      103 | Processing
(3 rows)
```

Display all users

```
postgres=# select * from users;
 u_id | name   | email
-----+-----+-----
     1 | Prachi | prachi@gmail.com
     2 | Arya   | arya@gmail.com
     3 | Hetvi  | hetvi@gmail.com
     4 | Priya  | priya@gmail.com
     5 | Shreya | Shreya@gmail.com
(5 rows)
```

Update Product Price

```
postgres=# UPDATE product
postgres=# SET price = 55000
postgres=# WHERE name = 'Laptop';
UPDATE 1
postgres=# select * from product;
```

p_id	name	price	description	c_id
2	Phone	20000.00	Smartphone	1
3	T-shirt	5000.00	Cotton	2
4	Headphone	2000.00	Wireless	1
1	Laptop	55000.00	Gaming Laptop	1

(4 rows)

Select with Compound Condition (AND)

```
postgres=# SELECT * FROM product
postgres=# WHERE price > 2000 AND c_id = 1;
```

p_id	name	price	description	c_id
2	Phone	20000.00	Smartphone	1
1	Laptop	55000.00	Gaming Laptop	1

(2 rows)

Find Products with Price Greater Than 1000

```
postgres=# select name, price from product where price > 1000;
```

name	price
Laptop	50000.00
Phone	20000.00
T-shirt	5000.00
Headphone	2000.00

(4 rows)

Query to fetch multiple Names from Users tables

```
postgres=# SELECT * FROM users
postgres=# WHERE name = 'Arya' OR name = 'Priya';
```

u_id	name	email
2	Arya	arya@gmail.com
4	Priya	priya@gmail.com

(2 rows)

Find Addresses in Maharashtra State

```
postgres=# select * from address where state='Maharashtra';
 a_id | u_id | city  | state  | country
-----+-----+-----+-----+-----
    1 |    1 | Mumbai | Maharashtra | India
    3 |    3 | Pune  | Maharashtra | India
(2 rows)
```

Retrieve Payments Made via UPI

```
postgres=# select * from payment where method='UPI';
 payment_id | order_no | method | amount
-----+-----+-----+-----
          1 |       101 | UPI    | 90000.00
(1 row)
```

Create index for faster Search

```
postgres=# CREATE INDEX idx_product_name
postgres=# ON product(name);
CREATE INDEX
postgres=# select * from product where name = 'Phone';
 p_id | name  | price  | description | c_id
-----+-----+-----+-----+-----
    2 | Phone | 20000.00 | Smartphone  |    1
(1 row)
```

Count Total Number of Users

```
postgres=# select count(*) from users;
 count
-----
      5
(1 row)
```

Find Maximum Product Price

```
postgres=# select MAX(price) from product;
 max
-----
50000.00
(1 row)
```

Display Users with Their Orders

```
postgres=# select u.name, o.order_no from users u JOIN orders o ON u.u_id = o.u_id;
 name | order_no
-----+-----
 Prachi | 101
 Arya | 102
 Hetvi | 103
(3 rows)
```

Trigger Implementation

```
yovi=# CREATE TABLE productcategory ( c_id INT PRIMARY KEY, name VARCHAR(100) );
ERROR: relation "productcategory" already exists
yovi=# CREATE OR REPLACE FUNCTION uppercase_category() RETURNS TRIGGER AS $$ BEGIN NEW.name := UPPER(NEW.name); RETURN NEW; END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
yovi=# CREATE TRIGGER trg_uppercase_category BEFORE INSERT OR UPDATE ON productcategory FOR EACH ROW EXECUTE FUNCTION uppercase_category();
CREATE TRIGGER
yovi=# INSERT INTO productcategory VALUES (4,'mobile');
INSERT 0 1
yovi=# SELECT * FROM productcategory;
 c_id | name
-----+-----
 1 | Electronics
 2 | clothing
 3 | Footwear
 4 | MOBILE
(4 rows)
```

List Products in Each Order

```
postgres=# select o.order_no, p.name FROM orders o JOIN order_product op ON o.order_no = op.order_no JOIN product p ON p.p_id = op.p_id;
 order_no | name
-----+-----
 101 | Laptop
 101 | Phone
 102 | T-shirt
 103 | Headphone
(4 rows)
```

Insert Record into Users Table

```
postgres=# INSERT INTO users (u_id, name, email)
postgres=# VALUES (6, 'Riya', 'riya@gmail.com');
INSERT 0 1
postgres=# select * from users;
 u_id | name | email
-----+-----+-----
 1 | Prachi | prachi@gmail.com
 2 | Arya | arya@gmail.com
 3 | Hetvi | hetvi@gmail.com
 4 | Priya | priya@gmail.com
 5 | Shreya | Shreya@gmail.com
 6 | Riya | riya@gmail.com
(6 rows)
```

Show Products with Their Categories

```
postgres=# select p.name, c.name FROM product p JOIN productCategory c ON p.c_id = c.c_id;
 name | name
-----+-----
 Laptop | Electronics
 Phone | Electronics
 T-shirt | Clothing
 Headphone | Electronics
(4 rows)
```

Delete Record from Cart Table

```
postgres=# DELETE FROM cart_product
postgres=# WHERE cart_id = 3;
DELETE 1
postgres=# DELETE FROM cart
postgres=# WHERE cart_id = 3;
DELETE 1
postgres=# select * from cart_product WHERE cart_id = 3;
  cart_id | p_id | quantity
-----+-----+-----
(0 rows)
```

Alter Table to Add New Column

```
postgres=# ALTER TABLE users
postgres=# ADD phone VARCHAR(15);
ALTER TABLE
postgres=# select * from users;
  u_id | name  | email           | phone
-----+-----+-----+-----
    1 | Prachi | prachi@gmail.com |
    2 | Arya   | arya@gmail.com   |
    3 | Hetvi  | hetvi@gmail.com  |
    4 | Priya  | priya@gmail.com  |
    5 | Shreya | shreya@gmail.com |
    6 | Riya   | riya@gmail.com   |
(6 rows)
```

Grant Permissions to user

```
postgres=# GRANT SELECT, INSERT ON product TO public;
GRANT
postgres=# select * from product;
  p_id | name      | price  | description  | c_id
-----+-----+-----+-----+-----
    2 | Phone     | 20000.00 | Smartphone   | 1
    3 | T-shirt   | 5000.00 | Cotton       | 2
    4 | Headphone | 2000.00 | Wireless     | 1
    1 | Laptop    | 55000.00 | Gaming Laptop | 1
(4 rows)
```

Display Order Tracking Status

```
postgres=# select o.order_no, t.status FROM orders o JOIN TrackingDetail t ON o.order_no = t.order_no;
 order_no | status
-----+-----
    101 | shipped
    102 | Delivered
    103 | Processing
(3 rows)
```

Revoke Permission from Users

```
postgres=# REVOKE INSERT ON product FROM public;
REVOKE
postgres=# select * from product;
```

p_id	name	price	description	c_id
2	Phone	20000.00	Smartphone	1
3	T-shirt	5000.00	Cotton	2
4	Headphone	2000.00	Wireless	1
1	Laptop	55000.00	Gaming Laptop	1

```
(4 rows)
```

Count Total Records in Table

```
postgres=# SELECT COUNT(*) FROM users;
count
-----
      6
(1 row)
```

Calculate Total Order Amount

```
postgres=# select SUM(order_amount) FROM orders;
sum
-----
91700.00
(1 row)
```

Find Average Product Price

```
postgres=# select AVG(price) FROM product;
avg
-----
19250.000000000000
(1 row)
```

Count Total Orders

```
postgres=# select COUNT(*) FROM orders;
count
-----
      3
(1 row)
```

Select with Compound Condition (NOT BETWEEN)

```
postgres=# SELECT * FROM orders
postgres=# WHERE order_amount NOT BETWEEN 1000 AND 50000;
 order_no | u_id | a_id | order_amount | order_date
-----+-----+-----+-----+-----
      101 |    1 |    1 |    90000.00 | 2026-03-01
      103 |    3 |    3 |     200.00 | 2026-03-03
(2 rows)
```

Add Foreign Key Constraint

```
postgres=# ALTER TABLE orders
postgres=# ADD CONSTRAINT fk_user
postgres=# FOREIGN KEY (u_id) REFERENCES users(u_id);
ALTER TABLE
postgres=# select * from orders;
 order_no | u_id | a_id | order_amount | order_date
-----+-----+-----+-----+-----
      101 |    1 |    1 |    90000.00 | 2026-03-01
      102 |    2 |    2 |     1500.00 | 2026-03-02
      103 |    3 |    3 |     200.00 | 2026-03-03
(3 rows)
```

Drop table from database

```
postgres=# DROP TABLE TrackingDetail;
DROP TABLE
postgres=#
```

Find Minimum Product Price

```
postgres=# select MIN(price) FROM product;
min
-----
2000.00
(1 row)
```

Calculate Total Quantity per Order

```
postgres=# select order_no, SUM(quantity) FROM order_product GROUP BY order_no;
 order_no | sum
-----+-----
      101 |    3
      103 |    1
      102 |    3
(3 rows)
```

Count Addresses by State

```
postgres=# select state, COUNT(*) FROM address GROUP BY state;
      state | count
-----+-----
Maharashtra |     2
Uttar Pradesh |     1
Karnataka   |     1
Delhi       |     1
(4 rows)
```

View Implementation

```
yovi=# CREATE VIEW category_view AS SELECT c_id, UPPER(name) AS name FROM productcategory;
CREATE VIEW
yovi=# SELECT * FROM category_view;
 c_id | name
-----+-----
    1 | ELECTRONICS
    2 | CLOTHING
    3 | FOOTWEAR
    4 | MOBILE
(4 rows)
```

Count Orders by Status

```
postgres=# select status, COUNT(*) FROM TrackingDetail GROUP BY status;
      status | count
-----+-----
Delivered   |     1
Processing  |     1
shipped     |     1
(3 rows)
```

Find Users Who Have Placed Orders

```
postgres=# select name from users where u_id IN (select u_id from orders);
 name
-----
Prachi
Arya
Hetvi
(3 rows)
```


Find Products Priced Above Average

```
postgres=# select name from product where price > (select AVG(price) from product);
name
-----
Laptop
Phone
(2 rows)
```

Find User Who Placed a Specific Order

```
postgres=# select name from users where u_id = (select u_id from orders where order_no = 101);
name
-----
Prachi
(1 row)
```

Find Products That Have Been Ordered

```
postgres=# select name from product where p_id IN (select p_id from order_product);
name
-----
Laptop
Phone
T-shirt
Headphone
(4 rows)
```

Find Users Who Have Not Placed Orders

```
postgres=# select name from users where u_id NOT IN (select u_id from orders);
name
-----
Priya
Shreya
(2 rows)
```

Find Order with Maximum Amount

```
postgres=# select * from orders where order_amount = (select MAX(order_amount) from orders);
 order_no | u_id | a_id | order_amount | order_date
-----+-----+-----+-----+-----
      101 |    1 |    1 |    90000.00 | 2026-03-01
(1 row)
```

Find Products Cheaper Than Maximum Price

```
postgres=# select name from product where price < (select MAX(price) FROM product);
name
-----
Phone
T-shirt
Headphone
(3 rows)
```

Find Users with Payments Above 2000

```
postgres=# select name from users where u_id IN (select o.u_id from orders o JOIN payment p ON o.order_no = p.order_no WHERE p.amount > 2000);
 name
-----
 Prachi
(1 row)
```

Find Products in Electronics Category

```
postgres=# select name from product where c_id = (select c_id from productCategory where name = 'Electronics');
 name
-----
 Laptop
 Phone
 Headphone
(3 rows)
```

Find Tracking Details for a Specific User

```
postgres=# select * from TrackingDetail where order_no IN (select order_no from orders where u_id = 1);
 t_id | order_no | status
-----+-----+-----
    1 |      101 | shipped
(1 row)
```

Count Orders for Each User (Including No Orders)

```
postgres=# select u.name, COUNT(o.order_no) from users u LEFT JOIN orders o ON u.u_id = o.u_id GROUP BY u.name;
 name | count
-----+-----
 Arya |    1
 Hetvi |    1
 Shreya |    0
 Priya |    0
 Prachi |    1
(5 rows)
```

Count Number of Times Each Product is Ordered

```
postgres=# select p.name, count(op.order_no) from product p LEFT JOIN order_product op ON p.p_id = op.p_id GROUP BY p.name;
 name | count
-----+-----
 Headphone |    1
 Phone |    1
 T-shirt |    1
 Laptop |    1
(4 rows)
```

Find Users with More Than One Order

```
postgres=# select u.name from users u JOIN orders o ON u.u_id = o.u_id GROUP BY u.name HAVING count(o.order_no) > 1;
 name
-----
(0 rows)
```

Sort Products by Price (Descending Order)

```
postgres=# select name from product ORDER BY price DESC;
name
-----
Laptop
Phone
T-shirt
Headphone
(4 rows)
```

Sort Orders by Date

```
postgres=# select * from orders ORDER BY order_date;
order_no | u_id | a_id | order_amount | order_date
-----+-----+-----+-----+-----
      101 |    1 |    1 |    90000.00 | 2026-03-01
      102 |    2 |    2 |     1500.00 | 2026-03-02
      103 |    3 |    3 |      200.00 | 2026-03-03
(3 rows)
```

Display First 3 Products

```
postgres=# select * from product LIMIT 3;
p_id | name      | price    | description      | c_id
-----+-----+-----+-----+-----
    1 | Laptop    | 50000.00 | Gaming Laptop    |    1
    2 | Phone     | 20000.00 | Smartphone       |    1
    3 | T-shirt   |  5000.00 | Cotton           |    2
(3 rows)
```

Find Unique States from Address Table

```
postgres=# select DISTINCT state from address;
state
-----
Maharashtra
Uttar Pradesh
Karnataka
Delhi
(4 rows)
```

Find Payments Between 1000 and 50000

```
postgres=# select * from payment where amount BETWEEN 1000 AND 50000;
 payment_id | order_no | method | amount
-----+-----+-----+-----
          2 |      102 |  card  | 1500.00
          3 |      103 |  cash  | 2000.00
(2 rows)
```

Find Users Whose Name Starts with 'P'

```
postgres=# select name from users where name LIKE 'p%';
 name
-----
(0 rows)
```

Calculate Total Quantity Sold per Product

```
postgres=# select p.name, SUM(op.quantity) AS total_sold FROM product p JOIN order_product op ON p.p_id = op.p_id GROUP BY p.name;
 name | total_sold
-----+-----
Headphone | 1
Phone | 2
T-shirt | 3
Laptop | 1
(4 rows)
```

9. Conclusion

The E-commerce database system designed in this project successfully demonstrates the effective use of DBMS concepts to manage and organize large volumes of data in a structured manner. By implementing a well-defined database design using the ER model, the system ensures proper handling of entities such as users, products, orders, and payments along with their relationships. The use of normalization techniques and constraints helps in maintaining data integrity, reducing redundancy, and improving overall system efficiency. This project highlights how a properly designed database can support smooth operations, enhance performance, and provide a reliable foundation for real-world E-commerce applications.

Key Achievements

1. Successfully designed a structured and normalized E-commerce database.
2. Established clear relationships between entities like users, products, and orders.
3. Reduced data redundancy and ensured data consistency.
4. Implemented data integrity using keys and constraints.
5. Improved data retrieval speed and overall system performance.